Cycle - 4

Familiarization of MongoDB commands

1. Create Database CollegeDB

   use CollegeDB

2. Create Collection named “Student”

   db.createCollection("students")

3. Insert 10 Records

```
db.students.insertMany([
{ name: "Arun", age: 22, course: "MCA" },
{ name: "Meena", age: 23, course: "MSc" },
{ name: "Rahul", age: 24, course: "MCA" },
{ name: "Sneha", age: 22, course: "MBA" },
{ name: "Vijay", age: 23, course: "MCA" },
{ name: "Anu", age: 21, course: "MSc" },
{ name: "Riya", age: 22, course: "MBA" },
{ name: "Kiran", age: 24, course: "MCA" },
{ name: "Deepa", age: 23, course: "MSc" },
{ name: "Manu", age: 22, course: "MBA" }
])
```

4. Display First 5 Records

   db.students.find().limit(5)

5. Skip First 3 Records

   db.students.find().skip(3)

6. Sort by Age (Ascending)

```
db.students.find().sort({ age: 1 })
```

7. Update a Record

```
db.students.updateOne(
  { name: "Arun" },
  { $set: { age: 25 } }
)
```

8. Delete a Record

```
db.students.deleteOne({ name: "Meena" })
```

9. Drop Collection

```
db.dropDatabase()
```

Program –

QUIRES:

a) Create a database Myclass.

```
use Myclass
```

b) Create a collection named "db_students" Should contain this fields: (student_name, student_rollno, mark[subject, mark]) Nb: Mark should be stored as array

```
db.createCollection('db_students')
```

c) Insert details of 10 students in a class

```
db.Myclass.insertMany([
{"_id" : ObjectId("62552df480b49718c39a5de4"), "student_name": "adarsh", "student_rollno": 1, "mark": [ { "dbms": 70 }, { "mpmc": 70 }, { "cn": 70 }, { "ss": 70 }, { "dm": 69 }, { "moss": 71 } ] },
```

{ "_id" : ObjectId("62552df480b49718c39a5de5"), "student_name": "amal", "student_rollno": 11, "mark": [{ "dbms": 50 }, { "mpmc": 60 }, { "cn": 60 }, { "ss": 100 }, { "dm": 69 }, { "moss": 70 }] },
{ "_id" : ObjectId("62552df480b49718c39a5de6"), "student_name": "joel", "student_rollno": 34, "mark": [{ "dbms": 80 }, { "mpmc": 80 }, { "cn": 80 }, { "ss": 90 }, { "dm": 69 }, { "moss": 100 }] },
{ "_id" : ObjectId("62552df480b49718c39a5de7"), "student_name": "athul", "student_rollno": 20, "mark": [{ "dbms": 84 }, { "mpmc": 50 }, { "cn": 60 }, { "ss": 96 }, { "dm": 69 }, { "moss": 65 }] },
{ "_id" : ObjectId("62552df480b49718c39a5de8"), "student_name": "dude", "student_rollno": 7, "mark": [{ "dbms": 74 }, { "mpmc": 60 }, { "cn": 70 }, { "ss": 86 }, { "dm": 69 }, { "moss": 55 }] },
{ "_id" : ObjectId("62552df480b49718c39a5de9"), "student_name": "pranav", "student_rollno": 60, "mark": [{ "dbms": 78 }, { "mpmc": 67 }, { "cn": 50 }, { "ss": 76 }, { "dm": 69 }, { "moss": 67 }] },
{ "_id" : ObjectId("62552df480b49718c39a5dea"), "student_name": "noel", "student_rollno": 15, "mark": [{ "dbms": 88 }, { "mpmc": 87 }, { "cn": 80 }, { "ss": 86 }, { "dm": 69 }, { "moss": 87 }] }, 153 154

{ "_id" : ObjectId("62552df480b49718c39a5deb"), "student_name": "nivin", "student_rollno": 40, "mark": [{ "dbms": 58 }, { "mpmc": 57 }, { "cn": 55 }, { "ss": 56 }, { "dm": 69 }, { "moss": 57 }] },
{ "_id" : ObjectId("62552df480b49718c39a5dec"), "student_name": "thomas", "student_rollno": 23, "mark": [{ "dbms": 98 }, { "mpmc": 97 }, { "cn": 95 }, { "ss": 96 }, { "dm": 69 }, { "moss": 97 }] },
{ "_id" : ObjectId("62552df480b49718c39a5ded"), "student_name": "athulp", "student_rollno": 21, "mark": [{ "dbms": 88 }, { "mpmc": 87 }, { "cn": 85 }, { "ss": 86 }, { "dm": 69 }, { "moss": 87 }] }
])

d) List the entire students in the class

```
db.Myclass.find()
```

e) Update mark of any one student in the collection "db_students"

```
db.db_students.updateOne({ student_name: "amal" },{ $set: { mark: ["en", 50] }
})
```

f) Delete the data of first student in the collection

```
db.coll.deleteOne({name:"amal"})
```

CYCLE 5

Aim: Familiarisation of MongoDB Commands.

The most popular NoSQL database, is an open-source document-oriented database. The term 'NoSQL' means 'non-relational'. Some of the commonly used functions are:

- **InsertMany():** It is used to insert multiple row of datas at the same time.
- **Find():** It is used to display all the elements of a document.
- **findOne():** It is used to display the first document of collection.
- **Sort():** Used to sort the elements while displaying.

1. Create a Mongodb Database named "Inventory".

   - **use Inventory**

```
> use Inventory
switched to db Inventory
```

2. Create a collection named 'Products' and Insert the following documents.

   - **db.products.insertMany([**
     { "_id" : 1, "name" : "xPhone", "price" : 799, "releaseDate": ISODate("2011-05-14"), "spec" : { "ram" : 4, "screen" : 6.5, "cpu" : 2.66},"color":["white","black"],"storage":[64,128,256]},
     { "_id" : 2, "name" : "xTablet", "price" : 899, "releaseDate": ISODate("2011-09-01") , "spec" : { "ram" : 16, "screen" : 9.5, "cpu" : 3.66},"color":["white","black","purple"],"storage":[128,256,512]},
     { "_id" : 3, "name" : "SmartTablet", "price" : 899, "releaseDate": ISODate("2015-01-14"), "spec" : { "ram" : 12, "screen" : 9.7, "cpu" : 3.66},"color":["blue"],"storage":[16,64,128]},
     { "_id" : 4, "name" : "SmartPad", "price" : 699, "releaseDate": ISODate("2020-05-14"),"spec" : { "ram" : 8, "screen" : 9.7, "cpu" : 1.66},"color":["white","orange","gold","gray"],"storage":[128,256,1024]},
     { "_id" : 5, "name" : "SmartPhone", "price" : 599,"releaseDate": ISODate("2022-09-14"), "spec" : { "ram" : 4, "screen" : 9.7, "cpu" : 1.66},"color":["white","orange","gold","gray"],"storage":[128,256]}])

```
> db.products.insertMany([
... { "_id" : 1, "name" : "xPhone", "price" : 799, "releaseDate": ISODate("2011-05-14"), "spec" : { "ram" : 4,
"screen" : 6.5, "cpu" : 2.66},"color":["white","black"],"storage":[64,128,256]},
... { "_id" : 2, "name" : "xTablet", "price" : 899, "releaseDate": ISODate("2011-09-01") , "spec" : { "ram" : 1
6, "screen" : 9.5, "cpu" : 3.66},"color":["white","black","purple"],"storage":[128,256,512]},
... { "_id" : 3, "name" : "SmartTablet", "price" : 899, "releaseDate": ISODate("2015-01-14"), "spec" :{ "ram" :
 12, "screen" : 9.7, "cpu" : 3.66},"color":["blue"],"storage":[16,64,128]},
... { "_id" : 4, "name" : "SmartPad", "price" : 699, "releaseDate": ISODate("2020-05-14"),"spec" : { "ram" : 8,
 "screen" : 9.7, "cpu" : 1.66},"color":["white","orange","gold","gray"],"storage":[128,256,1024]},
... { "_id" : 5, "name" : "SmartPhone", "price" : 599,"releaseDate": ISODate("2022-09-14"), "spec" : { "ram" :
4, "screen" : 9.7, "cpu" : 1.66},"color":["white","orange","gold","gray"],"storage":[128,256]}])
{ "acknowledged" : true, "insertedIds" : [ 1, 2, 3, 4, 5 ] }
```

3. Display all documents in the collection product.

- **db.products.find()**

```
> db.products.find()
{ "_id" : 1, "name" : "xPhone", "price" : 799, "releaseDate" : ISODate("2011-05-14T00:00:00Z"), "spec" : { "ram
" : 4, "screen" : 6.5, "cpu" : 2.66 }, "color" : [ "white", "black" ], "storage" : [ 64, 128, 256 ] }
{ "_id" : 2, "name" : "xTablet", "price" : 899, "releaseDate" : ISODate("2011-09-01T00:00:00Z"), "spec" : { "ra
m" : 16, "screen" : 9.5, "cpu" : 3.66 }, "color" : [ "white", "black", "purple" ], "storage" : [ 128, 256, 512
] }
{ "_id" : 3, "name" : "SmartTablet", "price" : 899, "releaseDate" : ISODate("2015-01-14T00:00:00Z"), "spec" : {
 "ram" : 12, "screen" : 9.7, "cpu" : 3.66 }, "color" : [ "blue" ], "storage" : [ 16, 64, 128 ] }
{ "_id" : 4, "name" : "SmartPad", "price" : 699, "releaseDate" : ISODate("2020-05-14T00:00:00Z"), "spec" : { "r
am" : 8, "screen" : 9.7, "cpu" : 1.66 }, "color" : [ "white", "orange", "gold", "gray" ], "storage" : [ 128, 25
6, 1024 ] }
{ "_id" : 5, "name" : "SmartPhone", "price" : 599, "releaseDate" : ISODate("2022-09-14T00:00:00Z"), "spec" : {
"ram" : 4, "screen" : 9.7, "cpu" : 1.66 }, "color" : [ "white", "orange", "gold", "gray" ], "storage" : [ 128,
256 ] }
```

4. Display all the details of product with _ id is 2.

- **db.products.find({_id: 2})**

```
> db.products.find({_id: 2})
{ "_id" : 2, "name" : "xTablet", "price" : 899, "releaseDate" : ISODate("2011-09-01T00:00:00Z"), "spec" : { "ra
m" : 16, "screen" : 9.5, "cpu" : 3.66 }, "color" : [ "white", "black", "purple" ], "storage" : [ 128, 256, 512
] }
```

5. Display the first document in the collection product.

- **db.products.findOne()**

```
> db.products.findOne()
{
        "_id" : 1,
        "name" : "xPhone",
        "price" : 799,
        "releaseDate" : ISODate("2011-05-14T00:00:00Z"),
        "spec" : {
                "ram" : 4,
                "screen" : 6.5,
                "cpu" : 2.66
        },
        "color" : [
                "white",
                "black"
        ],
        "storage" : [
                64,
                128,
                256
        ]
}
```

6. Display name and price of product with _ id is 5.

- **db.products.find({ _id: 5}, { name: 1,price: 1})**

```
> db.products.find({ _id: 5}, { name: 1,price: 1})
{ "_id" : 5, "name" : "SmartPhone", "price" : 599 }
```

7. Query the products collection to select all documents where the value of the price field equals 899.

- **db.products.find({price: {$eq: 899}}, {name: 1,price: 1})**

```
> db.products.find({
... price: {
... $eq: 899
... }
... }, {
... name: 1,
... price: 1
... })
{ "_id" : 2, "name" : "xTablet", "price" : 899 }
{ "_id" : 3, "name" : "SmartTablet", "price" : 899 }
```

8. Search for documents where the value of the ram field in the spec document equals 4.

- **db.products.find({"spec.ram": {$eq: 4}}, {name: 1,"spec.ram": 1})**

```
> db.products.find({
... "spec.ram": {
... $eq: 4
... }
... }, {
... name: 1,
... "spec.ram": 1
... })
{ "_id" : 1, "name" : "xPhone", "spec" : { "ram" : 4 } }
{ "_id" : 5, "name" : "SmartPhone", "spec" : { "ram" : 4 } }
```

9. Query the products collection to find all documents where the array color contains an element with the value "black".

- **db.products.find({color: {$eq: "black"}}, {name: 1,color: 1})**

```
> db.products.find({
... color: {
... $eq: "black"
... }
... }, {
... name: 1,
... color: 1
... })
{ "_id" : 1, "name" : "xPhone", "color" : [ "white", "black" ] }
{ "_id" : 2, "name" : "xTablet", "color" : [ "white", "black", "purple" ] }
```

10. Select documents in the products collection with the published date is 2020-05-14.

- **db.products.find({releaseDate: {$eq: new ISODate("2020-05-14")}}, {name: 1,releaseDate: 1})**

```
> db.products.find({
... releaseDate: {
... $eq: new ISODate("2020-05-14")
... }
... }, {
... name: 1,
... releaseDate: 1
... })
{ "_id" : 4, "name" : "SmartPad", "releaseDate" : ISODate("2020-05-14T00:00:00Z") }
```

11. Select documents from the products collection where price is less than 799.

- **db.products.find({price: {$lt: 799}}, {name: 1,price: 1})**

```
> db.products.find({
... price: {
... $lt: 799
... }
... }, {
... name: 1,
... price: 1
... })
{ "_id" : 4, "name" : "SmartPad", "price" : 699 }
{ "_id" : 5, "name" : "SmartPhone", "price" : 599 }
```

12. Select documents where the value of the screen field in the spec document is less than 7.

- **db.products.find({"spec.screen": {$lt: 7}}, {name: 1,"spec.screen": 1})**

```
> db.products.find({
... "spec.screen": {
... $lt: 7
... }
... }, {
... name: 1,
... "spec.screen": 1
... })
{ "_id" : 1, "name" : "xPhone", "spec" : { "screen" : 6.5 } }
```

13. Query the products collection to find all documents where the array storage has at least one element less than 128.

- **db.products.find({storage: {$lt: 128}}, {name: 1,storage: 1})**

```
> db.products.find({
... storage: {
... $lt: 128
... }
... }, {
... name: 1,
... storage: 1
... })
{ "_id" : 1, "name" : "xPhone", "storage" : [ 64, 128, 256 ] }
{ "_id" : 3, "name" : "SmartTablet", "storage" : [ 16, 64, 128 ] }
```

14. Display documents from the products collection whose the price is either 599 or 799.

- **db.products.find({price: {$in: [699, 799]}}, {name: 1,price: 1})**

```
> db.products.find({
... price: {
... $in: [699, 799]
... }
... }, {
... name: 1,
... price: 1
... })
{ "_id" : 1, "name" : "xPhone", "price" : 799 }
{ "_id" : 4, "name" : "SmartPad", "price" : 699 }
```

15. Display documents where the color array has at least one element either "black" or "white".

- **db.products.find({color: {$in: ["black", "white"]}}, { name: 1,color: 1})**

```
> db.products.find({
... color: {
... $in: ["black", "white"]
... }
... }, { name: 1,
... color: 1
...
... })
{ "_id" : 1, "name" : "xPhone", "color" : [ "white", "black" ] }
{ "_id" : 2, "name" : "xTablet", "color" : [ "white", "black", "purple" ] }
{ "_id" : 4, "name" : "SmartPad", "color" : [ "white", "orange", "gold", "gray" ] }
{ "_id" : 5, "name" : "SmartPhone", "color" : [ "white", "orange", "gold", "gray" ] }
```

16. Display documents from the products collection whose price is neither 599 or 799.

- **db.products.find({price: {$nin: [699, 799]}}, {name: 1,price: 1})**

```
> db.products.find({
... price: {
... $nin: [699, 799]
... }
... }, {
... name: 1,
... price: 1
... })
{ "_id" : 2, "name" : "xTablet", "price" : 899 }
{ "_id" : 3, "name" : "SmartTablet", "price" : 899 }
{ "_id" : 5, "name" : "SmartPhone", "price" : 599 }
```

17. Display documents where the color array doesn't have an element that is either "black" or "white".

- **db.products.find({color: {$nin: ["black", "white"]}}, {name: 1,color: 1})**

```
> db.products.find({
... color: {
... $nin: ["black", "white"]
... }
... }, {
... name: 1,
... color: 1
... })
{ "_id" : 3, "name" : "SmartTablet", "color" : [ "blue" ] }
```

18. Display all documents in the products collection where the value in the price field is equal to 899 and the value in the color field is either "white" or "black".

- **db.products.find({$and: [{price: 899}, {color: {$in: ["white", "black"]}}]}, {name: 1,price: 1,color: 1})**

```
> db.products.find({
... $and: [{
... price: 899
... }, {
... color: {
... $in: ["white", "black"]
... }
... }]
... }, {
... name: 1,
... price: 1,
... color: 1
... })
{ "_id" : 2, "name" : "xTablet", "price" : 899, "color" : [ "white", "black", "purple" ] }
```

19. Select all documents where the price is less than 699 or greater than 799.

- **db.products.find({$or: [{ price: {$lt: 699} },{ price: {$gt: 799} }]}, {name: 1price: 1})**

```
> db.products.find({
... $or: [
... { price: {$lt: 699} },
... { price: {$gt: 799} }
... ]
... }, {
... name: 1,
... price: 1
... })
{ "_id" : 2, "name" : "xTablet", "price" : 899 }
{ "_id" : 3, "name" : "SmartTablet", "price" : 899 }
{ "_id" : 5, "name" : "SmartPhone", "price" : 599 }
```

<u>Sorting Documents</u>

Use the sort() method to sort the documents by one or more fields. Specify { field: 1 } to sort documents by the field in ascending order and { field: -1 } to sort documents by the field in descending order. Use the dot notation { "embeddedDoc.field" : 1 } to sort the documents by the field in the embedded documents (embeddedDoc).

20. Sorts the products by the values in the ram field in the spec embedded documents. It includes the _id, name, and spec fields in the matching documents.

- **db.products.find({}, {name: 1,spec: 1}).sort({"spec.ram": 1});**

```
> db.products.find({}, {
... name: 1,
... spec: 1
... }).sort({
... "spec.ram": 1
... });
{ "_id" : 1, "name" : "xPhone", "spec" : { "ram" : 4, "screen" : 6.5, "cpu" : 2.66 } }
{ "_id" : 5, "name" : "SmartPhone", "spec" : { "ram" : 4, "screen" : 9.7, "cpu" : 1.66 } }
{ "_id" : 4, "name" : "SmartPad", "spec" : { "ram" : 8, "screen" : 9.7, "cpu" : 1.66 } }
{ "_id" : 3, "name" : "SmartTablet", "spec" : { "ram" : 12, "screen" : 9.7, "cpu" : 3.66 } }
{ "_id" : 2, "name" : "xTablet", "spec" : { "ram" : 16, "screen" : 9.5, "cpu" : 3.66 } }
```

21. Sorts the products by the values in the releaseDate field in descending order.

- **db.products.find({releaseDate: {$exists: 1}}, {name: 1,releaseDate: 1}).sort({releaseDate: -1});**

```
> db.products.find({
... releaseDate: {
... $exists: 1
... }
... }, {
... name: 1,
... releaseDate: 1
... }).sort({
... releaseDate: -1
... });
{ "_id" : 5, "name" : "SmartPhone", "releaseDate" : ISODate("2022-09-14T00:00:00Z") }
{ "_id" : 4, "name" : "SmartPad", "releaseDate" : ISODate("2020-05-14T00:00:00Z") }
{ "_id" : 3, "name" : "SmartTablet", "releaseDate" : ISODate("2015-01-14T00:00:00Z") }
{ "_id" : 2, "name" : "xTablet", "releaseDate" : ISODate("2011-09-01T00:00:00Z") }
{ "_id" : 1, "name" : "xPhone", "releaseDate" : ISODate("2011-05-14T00:00:00Z") }
```

22. Sort the products by name and price in ascending order. It selects only documents where the price field exists and includes the _id, name, and price fields in the matching documents.

- **db.products.find({'price': {$exists: 1}}, { name: 1, price: 1}).sort({price: 1,name: 1});**

```
> db.products.find({
... 'price': {
... $exists: 1
... }
... }, { name: 1, price: 1
... }).sort({
... price: 1,
... name: 1
... });
{ "_id" : 5, "name" : "SmartPhone", "price" : 599 }
{ "_id" : 4, "name" : "SmartPad", "price" : 699 }
{ "_id" : 1, "name" : "xPhone", "price" : 799 }
{ "_id" : 3, "name" : "SmartTablet", "price" : 899 }
{ "_id" : 2, "name" : "xTablet", "price" : 899 }
```

23. Get the most expensive product in the products collection. It includes the _id, name, and price fields in the returned documents.

- **db.products.find({}, {name: 1,price: 1}).sort({price: -1,name: 1}).limit(1);**

```
> db.products.find({}, {
... name: 1,
... price: 1
... }).sort({
... price: -1,
... name: 1
... }).limit(1);
{ "_id" : 3, "name" : "SmartTablet", "price" : 899 }
```